Introduction to InterSystems Package Manager

Version 2026.1
2026-04-20

InterSystems Corporation One Congress Street Boston MA 02114 www.intersystems.com

Introduction to InterSystems Package Manager
PDF generated on 2026-04-20
InterSystems IRIS Version 2026.1

For Support questions about any InterSystems products, contact:

InterSystems Worldwide Response Center (WRC)
Tel: +1-617-621-0700
Tel: +44 (0) 844 854 2917
Email: support@InterSystems.com

Table of Contents

Introduction to InterSystems Package Manager

This page describes how to install InterSystems Package Manager (IPM) and get started with it. It also summarizes commands needed for configuring IPM, discovering modules, managing modules, and other tasks.

1 Purpose

InterSystems Package Manager (IPM) is a package manager for InterSystems products. Use IPM to search, install, update, remove, and publish modules (also known as packages). Each module could be an application, framework, tool, or technology example. With IPM, you can install:

- InterSystems class definitions and ObjectScript routines
- Front end applications
- Interoperability applications
- IRIS BI solutions
- IRIS data sets
- Embedded Python wheels or any other files

The modules can reside in multiple locations, depending on how you want to share code:

- The InterSystems Community registry, at `https://pm.community.intersystems.com` (the default). This registry can be browsed only via the IPM tool.
- The Open Exchange, a community registry at https://openexchange.intersystems.com/, which contains code and libraries not written or supported by InterSystems.
- Private registries
- Local directories

Initial setup code for IPM is provided as part of your InterSystems product distribution. Once you load and use this code, you can upgrade IPM, retrieving the most recent version of the code from the default registry.

2 Getting Started

To install IPM and start using it:

1. Open an ObjectScript shell.
2. If you want to enable IPM only in a specific namespace, switch to that namespace.

   You can later enable it in other namespaces, if you change your mind.
3. Enter the following command:

ObjectScript

```
do $system.OBJ.Load($System.Util.InstallDirectory()_"dist/install/misc/zpm.xml","ck")
```

This loads the file *install-dir*/dist/install/misc/zpm.xml.

You will see a series of messages as the installer extracts items and then loads and compiles code. The process can take a couple of minutes.

4. Enter **zpm** to launch the shell. The prompt looks something like this:

```
=============================================================================
|| Welcome to the Package Manager Shell (ZPM). Version: 0.10.2             ||
|| Enter q/quit to exit the shell. Enter ?/help to view available commands ||
|| No registry configured                                                  ||
|| System Mode: <unset>                                                    ||
|| Mirror Status: NOTINIT                                                  ||
=============================================================================
IRIS for Windows <version information here>
zpm:USER>
```

You can now enter IPM shell commands. Notice that this message indicates that no registry is configured, so the next step is generally to configure IPM to use the default registry (https://pm.community.intersystems.com).

5. Enter one of the following commands, depending on whether you want to configure IPM in all namespaces:

   - Use `repo -reset-defaults` to configure IPM to use the default registry.
   - Use `repo community` to configure IPM to use the default registry and you also enable IPM in all namespaces.

   This generates additional messages including these:

```
registry
        Source:                         https://pm.community.intersystems.com
        Enabled?                        Yes
        Available?                      Yes
        Use for Snapshots?              Yes
        Use for Prereleases?            Yes
        Is Read-Only?                   No
        Default Deployment Registry?    No
```

6. Optionally enter the command `install zpm` if you want to install the most recent version of IPM.

   The most recent version of IPM requires Python 3.10, 3.11, or 3.12.

7. Now you can use the **search** command, which displays a list of packages that are available to install. The search command accepts wildcards. For example:

```
zpm:USER>search *data*
registry https://pm.community.intersystems.com:
ccd-data-profiler             1.1.3
databasesizemonitoring        1.0.0
dataset-apachelog             1.0.5
dataset-countries             1.1.4
dataset-finance               1.0.8
dataset-health                1.2.1
dataset-medical               1.0.5
dataset-oex-reviews           1.0.0
dataset-simple-m-n            1.0.5
dataset-titanic               1.0.0
iris-data-analysis            1.0.0
iris-datapipe                 2.0.5
iris-dataviz                  0.0.1
iris-energy-isodata           1.1.0
iris-rest-api-databasemanager 1.0.0
iristestdatagenerator         1.0.0
test-data                     1.0.4
```

8. You may want to try using IPM to download and install a package. If so, make sure that you are in the namespace where you want the code to reside. Then use the following command in the IPM shell:

   ```
   install modulename
   ```

   Where *modulename* is the name of a module that is available to you (as seen by the **search** command). You will see a series of messages, that vary depending on which module is being installed. The downloaded files are in *<install-dir>*/ipm/*modulename*.

   The *modulename* directory has a subdirectory for the version that you downloaded. That subdirectory should contain a README.md file or other explanatory file. The module.xml file defines the module. Other directories and files vary by module.

3 Overview of Module Definitions

At a high level, use the following general process to define a module:

1. Identify the code and related artifacts that you want to package together and organize the files into a suitable directory structure. The goal is to have one directory that completely contains all the needed files.

   Typically, the code source files are in different subdirectories depending on the language used, and by the type of code as well (classes versus routines, for example). Related artifacts such as exported globals or other data files are generally contained in other directories.

   The directory structure may need to be aligned with the directory structure used by your source control integration.

2. Within this directory structure, create an IPM manifest file (module.xml). This file describes what the module is, what it depends on, what code and resources it installs, and what to do before and after installation, optionally including running tests.

   There are two ways to start creating this file:

   - Copy the module.xml file from an existing package that looks similar to what you need.
   - Use the **generate** command, which prompts you for input and then generates a module.xml file.

   Make sure that you understand the dependencies of the new module and declare them in the manifest. This includes both the system dependencies (such as the InterSystems IRIS version or the OS version) and dependencies on any other modules.

3. Use the **load** command to load the module into your local instance, specifying the location of the manifest file. For example:

   ```
   zpm:USER>load C:\InterSystems\IRISTest\ipm\mysample\1.0.0
   ```

4 Commands for IPM Configuration

The following commands configure how IPM works:

- **init** – Configures the namespace for use of IPM (interactive). This sets up the local cache and allows for configuration of extensions for source control and IPM itself. Use this, for example, if you add a new namespace to the server.
- **config** – Updates IPM settings.

- **enable** – Enables IPM in other namespaces from a namespace in which IPM is already installed.
- **repo** – Configures the current namespace to search for modules on a remote server or on the local file system.
- **default-modifiers** – Manages default modifiers to use for all package manager commands in the current namespace.
- **unmap** – Remove IPM code and repository mappings in specified namespaces.

5 Commands for Discovery

The following commands enable you to obtain information about modules:

- **search** – Lists modules in the current registry.
- **list-installed** – Lists installed modules in the current namespace.
- **list-dependents** – Lists modules dependent on a specified module.
- **namespace** – Switches to a namespace and lists the modules installed in that namespace. There is also an option to list modules in all namespaces.
- **orphans** – Lists resources in the default code database (of the current namespace) that are not part of any module.

6 Commands for Module Lifecycle and Management

The following commands apply to the process of installing, compiling, testing, packaging, and publishing modules:

- **module-action** – Performs operations on modules: compiling, running tests, packaging, registering, and so on). Many of the other commands in this list are synonyms for **module-action** used with different keywords.
- **install** – Installs a module from a configured repository.
- **reinstall** – Reinstalls a module from a repository.
- **update** – Updates an installed module to a newer version and runs all the update steps needed.
- **reload** – Reloads module source into the namespace, without recompiling it.
- **compile** – Compiles resources in a module.
- **makedeployed** – Marks resources as deployed, where possible.
- **module-version** – Displays or modifies the version of a module.
- **test** – Runs unit tests for a module, if defined.
- **verify** – Runs the unit tests that are specified to run in the `verify` phase.
- **package** – Exports the module's resources and bundles them into a module artifact (.tgz file).
- **generate** – Generates a stub module.xml for use as a starting place for a new module manifest.
- **arrange** – Rearranges the resources in the given module manifest to follow the standard format.
- **publish** – Uploads the module to the repository for which deployment is enabled. Currently, there may only be one of these per namespace.
- **unpublish** – Removes a module from a repository.

- **uninstall** – Uninstalls a module that is currently installed locally.

7 Other Commands

The rest of the commands are as follows:

- **help** – Shows help for the IPM shell or for an individual IPM command.
- **version** – Prints the version numbers of the currently installed IPM and current registry.
- **run-from-file** – Executes a sequence of IPM commands from a text or JSON script.
- **load** – Loads a module from the specified directory or archive into the current namespace. Dependencies are also loaded automatically, provided that they can be found in repositories configured with the **repo** command. This command (and **import**) are useful when working with files that are not yet in any repo.
- **import** – Imports classes from a file or file(s), reexporting to source control if needed.
- **exec** – Executes the given ObjectScript statement and displays any output from it. This IPM option enables you to accomplish additional tasks without having to exit the IPM shell.
- **quit** – Exits the IPM shell.

8 See Also

- IPM Wiki pages on GitHub
- The IPM code on GitHub
- Where to report issues on GitHub